

UV : NFE204

GOTTE-MOUKOUYOU

e-mail : m.gotte@gmail.com

e-mail : moukouyou@gmail.com

Présentation et application d'ElasticSearch et de Kibana au domaine de l'assurance dans une optique Big Data

Sommaire

I) Contexte et Intérêt du projet

II) Description des données

III) Description de la solution technique

- A) Schéma d'architecture technique*
- B) Descriptions et versions des logiciels*
- C) Installation d'ElasticSearch*
- D) Installation de Kibana*

III) Présentation d'ElasticSearch

- A) Justification du choix d'une solution NoSQL*
- B) Architecture générale d'ElasticSearch*
- C) Présentation du processus d'indexation et de recherche dans ElasticSearch*
- D) Comparaison ElasticSearch - Solr*
- E) Injection des données dans ElasticSearch : Rivers*
- F) L'analyse des données avec Kibana*
- G) Implémentation des facettes*

IV) Percolation

IV) Scalabilité

IV) Sécurité

I) Contexte et Intérêt du projet

Le projet a lieu au sein de la direction internationale d'une compagnie d'assurance : Allianz. Celle-ci souhaiterait étudier les montants de primes et de sinistres des contrats collectifs en frais de santé et faire des tableaux de bord afin de les rendre disponibles par tous les services pour analyser. Les sinistres sont les montants payés par l'assureur lorsqu'un assuré souhaite se faire rembourser ses frais de soins. Les primes sont les cotisations que les assurés payent aux assureurs.

Le but est de mieux comprendre, visualiser les montants de primes et de sinistres des contrats d'assurances afin d'aider à la prise de décisions (indexation des contrats, identification des dérives des contrats d'assurances en frais de santé).

Les données viennent du système de gestion et concernent des organisations internationales comme l'OTAN, l'Agence Spatiale Européenne, la Banque Centrale Européenne ...

Les fichiers sont au format csv.

Les flux de données se caractérisent par des fichiers de mois de comptabilisation des primes et des sinistres. Ainsi par exemple pour l'année 2014, il y aura douze fichiers.

Toutes ces données csv seront importées dans Elasticsearch, un système de stockage NoSQL.

II) Description des données

Les données se présentent sous la forme de fichiers csv qui sont remontés tous les mois ; il y a deux types de fichiers, l'un pour les primes et l'autre pour les sinistres.

- Les fichiers des sinistres se nomment 03-MARS2014-ENC-AWC.csv, ces fichiers indiquent les sinistres qui ont été payés au mois de mars 2014.
- Les fichiers des primes se nomment 03-MARS2014-ENC-AWC.csv, ces fichiers indiquent les primes qui ont été encaissées par Allianz au mois de mars 2014.

Sur le fichier des primes voici le descriptif des variables importantes pour notre travail et l'analyse :

Nom de la variable	Exemple	Explication des données
tutpaq	0672	Le tuteur paquet est un regroupement de contrats
annee	2014	L'année de comptabilisation de la cotisation
mois	12	Le mois de comptabilisation de la cotisation
contrat	910009/001	Le numéro de contrat
devise	200	Devise de paiement : 200 euro, 072 : dollar, 061 : DKK (couronne danoise), 062 NOK (couronne norvégienne), 083 : GBP
Risque	5	La garantie sur laquelle est encaissée la cotisation, ici nous ne traiterons que de la santé donc du 5
TYPE FLUX	210	Ici nous ne traiterons que les montants bruts ie incluant les différents frais
mont_prime	4274,3	Le montant de la cotisation payée
perdeb	01/03/2014	Début de la période à laquelle est rattachée la cotisation
perfin	31/03/2014	Fin de la période à laquelle est rattachée la cotisation
DTSURANN	2014	L'année de la période à laquelle est rattachée la cotisation
DTSURMOI	12	Le mois de la période à laquelle est rattachée la cotisation

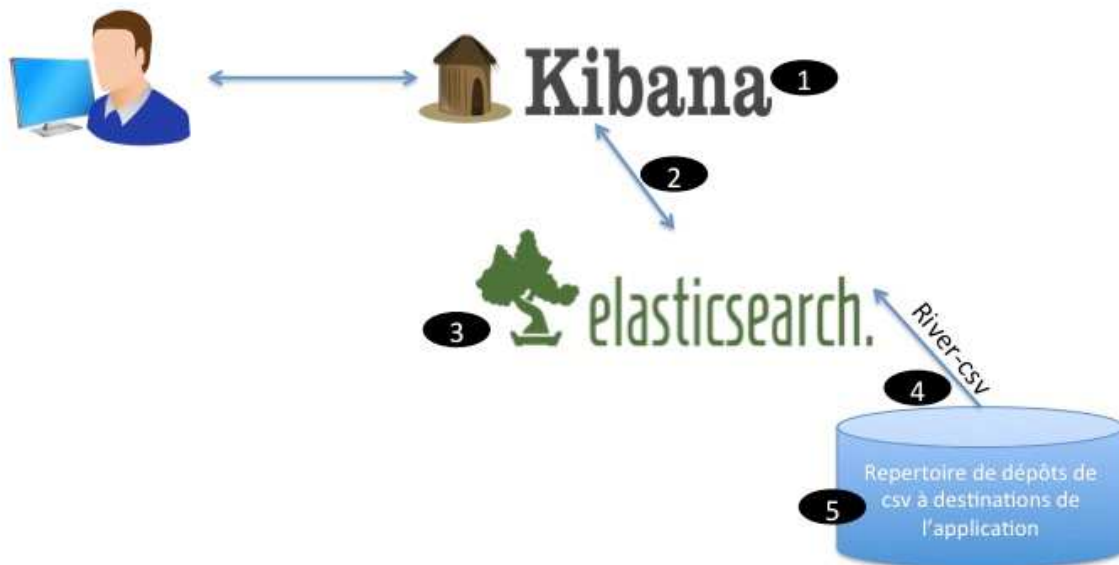
Sur le fichier des sinistres voici le descriptif des variables importantes pour notre travail et l'analyse :

Nom de la variable	Exemple	Explication
contrat	000874/000	Numéro du contrat
debind	20140901	Début de la période de comptabilisation du sinistre
finind	20140930	Fin de la période de comptabilisation du sinistre
DTCPTANN	2014	L'année de comptabilisation du sinistre
DTCPTMOI	11	Le mois de comptabilisation du sinistre
exesur	20020101	La date de survenance du sinistre
DTSURANN	2012	L'année de survenance du sinistre
montsin	301,24	Montant du sinistre
numass	154865558	Le numéro de l'assuré
nomass	ZZZZZZZ	Le nom de l'assuré
nomben	154878	Le numéro du bénéficiaire
devise	200	Devise de paiement : 200 euro, 072 : dollar, 061 DKK, 062 NOK, 083 : GBP
rsr	5D1	Le type de garantie, ici on ne traitera que de la santé
tpg	4306	Le tuteur paquet est un regroupement de contrats

III) Description de la solution technique

A) Schéma d'architecture technique

1. **Ihm Kibana** pour la visualisation des données côté front.
2. **Kibana** se connecte à **elasticsearch** grâce à son point d'accès REST
3. **Elasticsearch**: en tant que bdd NoSQL et moteur de recherche
4. **River-csv**: qui permet d'alimenter l'application à partir de fichiers csv.
5. **Répertoire réseau** (accessible aussi via ftp) , protégé, où seront déposés les fichiers csv consommés par l'application.



B) Descriptions et versions des logiciels

Voici une présentation des composants logiciels principaux de la solution :

- Kibana : version 4.0.0

Kibana est une interface hautement scalable, conçue nativement pour fonctionner avec Elasticsearch dans le but d'effectuer des recherches, construire des graphes bref, valoriser les données. L'aspect front de notre solution sera maintenu par Kibana.

- Elasticsearch : version 1.4.4

Base de données NoSQL et moteur de recherche distribué basé sur lucene.

- River-csv : version 2.1.1

Plugin ElasticSearch est chargé d'alimenter un index à partir d'une source de données csv, à intervalles réguliers. Il faut, dans sa configuration, préciser l'index ainsi que le type de documents ciblés par la « rivière ».

Nous nous étalerons sur ce sujet dans le chapitre dédié à la présentation des rivers.

C) Installation d'ElasticSearch

ElasticSearch est une application Java autonome, elle peut être facilement lancée à partir d'une ligne de commande.

Dans un premier temps, il faut télécharger ElasticSearch via le lien suivant :

<http://www.elasticsearch.org/overview/elkdownloads/> le fichier zip.

L'installation doit se faire de la manière suivante : Il faut vérifier que la dernière version Java 1.6.0_25 est installée.

Une fois le fichier décompressé, il y aura trois dossiers :

- Le répertoire bin qui contient le script de lancement Elasticsearch et le script d'installation de plugins
- Le répertoire config qui contient les fichiers de configuration d'ElasticSearch (elasticsearch.yml) et celui des logs (logging.yml)
- Le répertoire lib qui contient certaines bibliothèques utilisées par ElasticSearch.

Pour l'installation, il suffit de cliquer sur le fichier Elasticsearch.bat qui se trouve dans le dossier bin.

Sur linux, il faut lancer la commande \$./bin/elasticsearch

Pour pouvoir y accéder, il faut entrer l'url suivante <http://localhost:9200>

Si l'installation s'est bien déroulée, l'affichage doit être :

```
{
  "ok": true,
  "status": 200,
  "name": "Elsie-Dee",
  "version": {
    "number": "0.19.11",
    "snapshot_build": false
  },
  "tagline": "You Know, for Search"
}
```

Après cela, nous pouvons installer le plugin ElasticSearch-Head, il s'agit d'une interface d'administration d'ElasticSearch.

Une fois installé, il faut taper l'url suivante : http://localhost:9200/_plugin/head/

Cela va permettre de visualiser les clusters d'ElasticSearch. Ci-dessous, en voici un exemple.



Après le démarrage d'ElasticSearch :

- Le répertoire data est créé : il contient les documents indexés
- Le répertoire logs qui contient les logs

Grâce au plugin Head, on peut facilement visualiser les données et même faire des recherches une fois les données indexées.

Recherche sur 5 des 5 shards. 46502 résultats. 0.024 secondes											
_index	_type	_id	_score	origine	organismes	devise	contrat	ga	aperi	perdeb	
primes-allianz	primes	90aa686d-df05-46ef-94ac-b37b6de31925	1	04		061	910001/602	1	817	01092014	
primes-allianz	primes	bdad42c4-53c6-469c-beb4-10acc3c176ea	1	04		062	910001/103	1	817	01092014	
primes-allianz	primes	c7da9aa6-06c2-4ef2-985d-b1eb4e3a08aa	1	04		062	910001/103	1	817	01102014	
primes-allianz	primes	6e7c706a-afdb-4118-a762-e133d41d0127	1	04		062	910001/603	1	817	01092014	
primes-allianz	primes	ccaa3793-c0e8-4662-a57e-a0791b1526ee	1	03	1101	072	000874/041	1	32	01082014	
primes-allianz	primes	a9977043-04a2-4820-8d49-297c4321d835	1	04		072	910008/200	1	817	06082014	
primes-allianz	primes	a0c60e7a-d3a3-438f-93df-9a72e6a33509	1	04		072	910008/200	1	817	15082014	
primes-allianz	primes	391a7b1c-e4e0-4757-9836-fa745936f922	1	04		072	910008/200	1	817	16082014	
primes-allianz	primes	374498e9-0811-4ce7-9f38-a38ef90d804b	1	04		072	910008/200	1	817	18082014	
primes-allianz	primes	852c1346-9468-47a3-82be-753be88f7ddf	1	04		072	910008/200	1	817	03092014	
primes-allianz	primes	feb8a7ad-c70e-4426-86d5-77178a992338	1	04		072	910008/200	1	817	19092014	
primes-allianz	primes	0657c857-7646-4531-a5d4-a32ef99524d	1	04		072	910008/200	1	817	20092014	
primes-allianz	primes	03221ae1-6111-4cde-9260-4b15aa7b6984	1	04		072	910008/200	1	817	25092014	
primes-allianz	primes	cdadcd7c-b480-4f27-9caa-91c9caea81ea	1	04		072	910008/200	1	817	27092014	
primes-allianz	primes	235b7fea-c074-4fb3-b069-1050d130ac5d	1	04		072	910008/200	1	817	01102014	
primes-allianz	primes	59188c4e-bce9-443a-a4d3-7cf0552b4290	1	03	4066	077	909372/000	1	32	01082014	
primes-allianz	primes	753e62fb-9a73-458d-b50a-91f6968f7b09	1	03	4353	077	909566/000	1	32	01082014	
primes-allianz	primes	33f873f5-8d3a-4da3-b935-da0a066b91cc	1	04		083	910002/006	1	817	01102014	
primes-allianz	primes	6bc3ae76-94d3-4684-a608-29f4e4d3285d	1	04		083	910001/506	1	817	01102014	
primes-allianz	primes	11de48af-2a84-466a-93ed-2eb0f94b217f	1	03	1121	200	000874/021	1	32	01082014	
primes-allianz	primes	a7fa2d79-85ef-4bf4-82ff-9f3adb4d4d25	1	03	1131	200	000874/031	1	32	01082014	

D) Installation de Kibana

L'installation de Kibana se fait simplement en téléchargeant le fichier zip sur <https://download.elasticsearch.org/kibana/kibana/kibana-4.0.0-windows.zip>

Une fois décompressé, il faut ouvrir une fenêtre de commande, se placer dans le dossier bin, puis de l'exécuter par la commande `./bin/kibana`

Pour visualiser les données à partir de Kibana, il faudra aller, à partir de son navigateur, à l'adresse suivante par défaut : <http://localhost:5601>

III) Présentation d'ElasticSearch

ElasticSearch est un moteur de recherche Open source basé sur la librairie Apache Lucene.

Ce logiciel permet l'établissement d'un système distribué de nœuds et de clusters ainsi que le stockage et la recherche de très gros volumes de données.

Les données intégrées à Elasticsearch peuvent provenir d'une base relationnelle (par exemple Oracle) comme d'une base NoSQL.

A) Justification du choix d'une solution NoSQL

Nous souhaitons, autant que faire se peut, disposer d'une solution offrant les fonctionnalités suivantes :

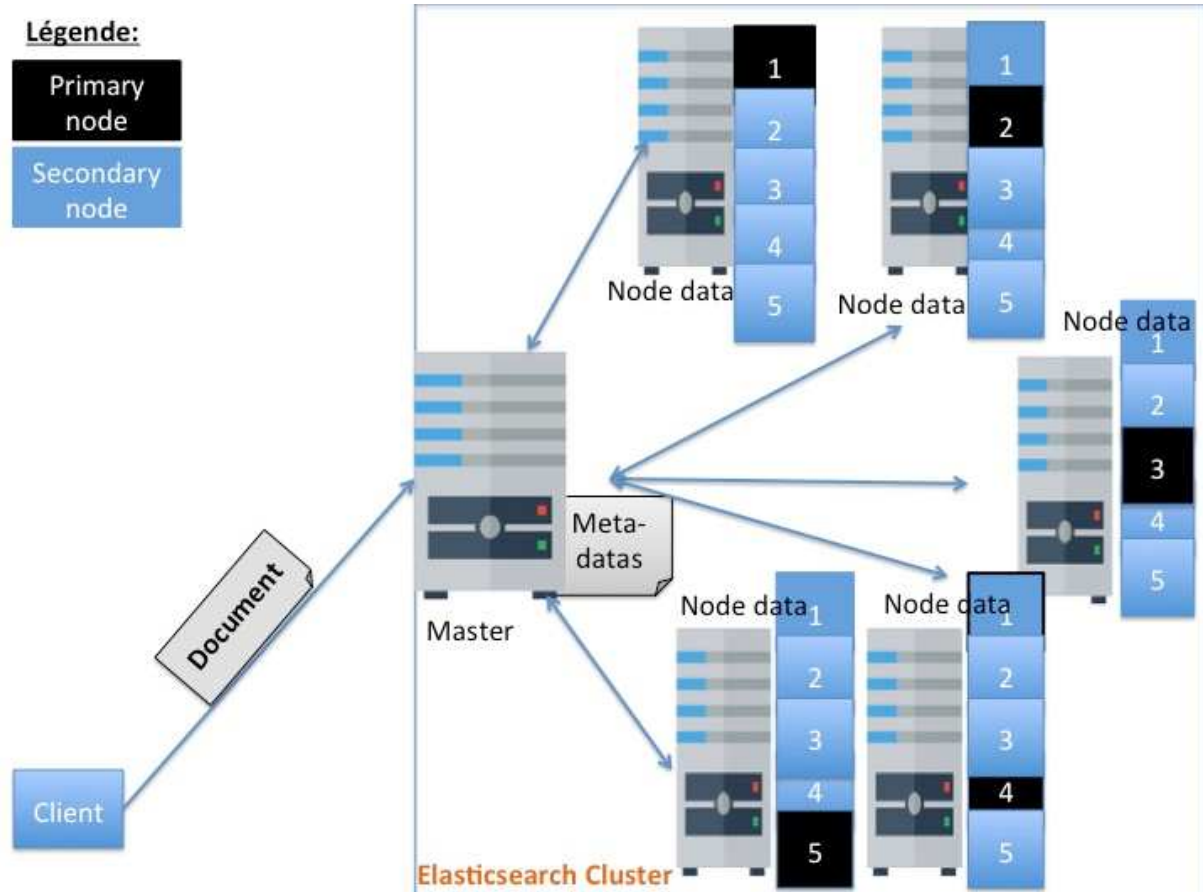
- stockage de données
- analyse de données
- visualisation et fouille de données

La solution retenue doit, en plus d'offrir les fonctionnalités citées ci-dessus, être hautement scalable ainsi que hautement disponible.

Le couple Elasticsearch et Kibana nous a semblé être celui qui répondait le mieux à toutes ces contraintes. Cela, compte tenu aussi, il convient de le souligner, de la facilité de sa mise en œuvre. Il s'agit, presque, d'une solution clé en main et gratuite (open source).

B) Architecture générale d'ElasticSearch

Architecture physique d'un cluster ElasticSearch



Nous détaillerons la configuration physique, précise, de la solution dans la section dédiée à la haute disponibilité. Ici, nous nous limiterons à la présentation du type de nœuds qui seront présents au sein du cluster qui hébergera la solution.

- Nœuds de type « master »: ceux-ci seront configurés (fichier `elasticsearch.yml`) avec les options suivantes

```
node.data : false
node.master : true
http.enabled : true
```

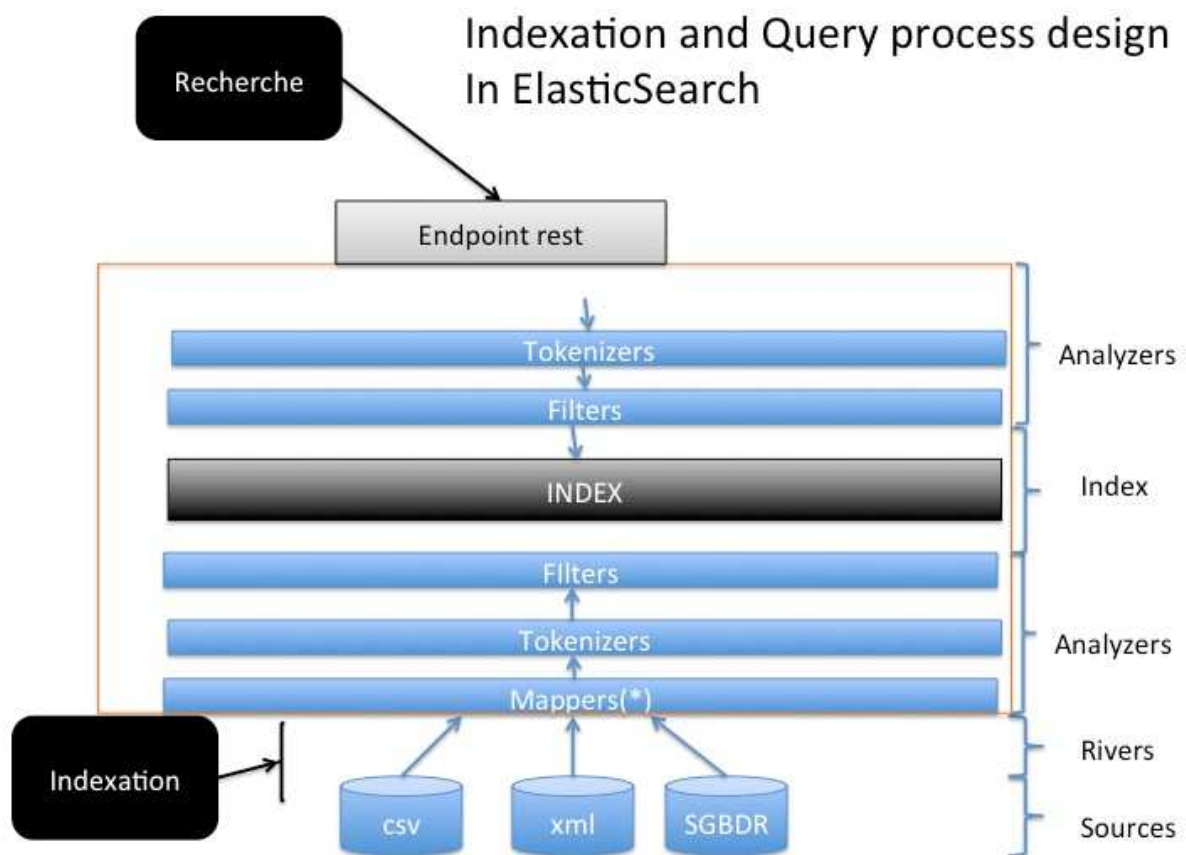
Ces nœuds n'hébergeront donc pas de données. Leurs seules responsabilités seront de maintenir les informations liées au bon fonctionnement (table de routage par exemple) du cluster ainsi que sa coordination. Par ailleurs, ils seront en charge du traitement des requêtes http.

Nœuds de type « data » : ils seront configurés comme suit :

```
node.data : true  
node.master : false  
http.enabled : false
```

Ainsi que le suggère la configuration, ils seront dédiés à l'hébergement des données de l'application.

C) Présentation du processus d'indexation et de recherche dans ElasticSearch



La configuration des nœuds, des shards et des réplicas.

Il faut ouvrir le fichier dans le dossier /config/elasticsearch.yml via le bloc note et changer comme on le souhaite les paramètres suivants :

- **cluster.name : Le nom de la grappe**
Le cluster est un ensemble de nœuds
- **index.number_of_shards : ici mettre le nombre de partitions voulu**
Les partitions correspondent à la partition élue principale dans l'ensemble du cluster. Il n'y en a qu'une seule par shard dans l'ensemble du cluster cela va permettre de découper un index en plusieurs parties pour y distribuer les documents
- **index.number_of_replicas: Ici mettre le nombre de copies voulu**

La réplication recopie une partition en une ou plusieurs copies dans l'ensemble du cluster

D) Comparaison Elasticsearch - Solr

Il existe, sur le marché des moteurs de recherche, une guerre qui ne dit pas son nom entre deux produits : Elasticsearch et Solr.

Ce qui suit, consistera surtout à expliquer ce qui nous a conduits, après étude des deux moteurs, à choisir Elasticsearch plutôt que Solr. Nous avons choisi d'axer cette explication autour de quatre axes qui sont :

La mise en œuvre

On dit souvent qu'ElasticSearch est installable très facilement. Comme décrit plus haut, les seules actions qui sont réellement requises consistent en la nécessité de donner un nom à notre cluster, de définir un nombre de shards et de replicas. Pour le reste, Elasticsearch s'occupe très bien du reste tout seul.

L'indexation :

Là aussi, Elasticsearch l'a emporté parce qu'il n'oblige pas à définir un schéma avant de procéder à une indexation.

Cela dit, il faut reconnaître que solr offre désormais la même possibilité depuis sa version 4.4. Il semble cependant légitime de penser que cet aspect est mieux géré par Elasticsearch du fait de la maturité de son implémentation au sein de ce produit.

La recherche :

Une des fonctionnalités qui nous intéressent très particulièrement est la recherche par facette. Très récemment, Elasticsearch a livré un nouveau module appelé agrégation. Avec la sortie de ce dernier, ils ont annoncé l'obsolescence prochaine des facettes.

Formatage des données

Avant d'injecter, nous devons créer deux index l'un pour les primes et l'autre pour les sinistres.

Nous créerons l'index sinistres-allianz partir de la commande :

```
curl -X PUT localhost:9200/elasticsearch/sinistres-allianz
```

puis l'index primes-allianz à partir de la commande :

```
curl -X PUT localhost:9200/elasticsearch/primes-allianz
```

Une fois ces deux index créés, il nous faut faire un mapping de chacune de ces variables car, par défaut, les données injectées sont de type caractère. Par conséquent, nous devons définir le type des variables avant l'injection des données.

Pour cela, nous devons exécuter la commande afin de définir le type de variable pour l'index allianz-sinistres :

POST sinistres-allianz/_mapping/sinistres

```
{
  "sinistres": {
    "properties": {
      "origine": {
        "type": "integer"
      },
      "organism": {
        "type": "string"
      },
      "contrat": {
        "type": "string"
      },
      "ga": {
        "type": "integer"
      },
      "compagn": {
        "type": "string"
      },
      "typmvt": {
        "type": "integer"
      },
      "topanul": {
        "type": "string"
      },
      "debind": {
        "type": "string"
      },
      "finind": {
        "type": "string"
      },
      "DTCPTANN": {
        "type": "integer"
      },
      "DTCPTMOI": {
        "type": "integer"
      },
      "datfge": {
        "type": "string"
      },
      "datevt": {
        "type": "string"
      },
      "exesur": {
        "type": "string"
      },
      "DTSURANN": {
        "type": "integer"
      }
    }
  },
}
```

```
"numecr": {
  "type": "integer"
},
"quotep": {
  "type": "integer"
},
"montsin": {
  "type": "double"
},
"numass": {
  "type": "string"
},
"nomass": {
  "type": "string"
},
"nomben": {
  "type": "string"
},
"topano": {
  "type": "string"
},
"dtresor": {
  "type": "string"
},
"devise": {
  "type": "integer"
},
"rsr": {
  "type": "string"
},
"tiers": {
  "type": "integer"
},
"tpg": {
  "type": "integer"
},
"aaaamm": {
  "type": "string"
},
"ref_num": {
  "type": "string"
},
"pays": {
  "type": "integer"
},
"annee": {
  "type": "integer"
},
"mois": {
  "type": "integer"
},
"TAUX": {
```

```

        "type": "string"
    },
    "monsinc": {
        "type": "double"
    }
}
}
}
}

```

Pour cela, nous devons exécuter la commande afin de définir le type de variable pour l'index **allianz-primes** :

POST primes-allianz/_mapping/primes

```

{
  "primes": {
    "properties": {
      "origine": {
        "type": "integer"
      },
      "organis": {
        "type": "string"
      },
      "devise": {
        "type": "integer"
      },
      "contrat": {
        "type": "string"
      },
      "ga": {
        "type": "integer"
      },
      "aperi": {
        "type": "string"
      },
      "perdeb": {
        "type": "string"
      },
      "perfin": {
        "type": "string"
      },
      "DTSURANN": {
        "type": "integer"
      },
      "DTSURMOI": {
        "type": "integer"
      },
      "rsr": {
        "type": "string"
      },
      "mont_prime": {
        "type": "double"
      }
    }
  },

```

```

    "tiers": {
      "type": "string"
    },
    "ref_date": {
      "type": "string"
    },
    "ref_num": {
      "type": "string"
    },
    "tutpaq": {
      "type": "integer"
    },
    "top": {
      "type": "string"
    },
    "annee": {
      "type": "integer"
    },
    "mois": {
      "type": "integer"
    },
    "TAUX": {
      "type": "string"
    },
    "mtprimec": {
      "type": "double"
    }
  }
}

```

E) Injection des données dans ElasticSearch : Rivers

Les rivers sont des connecteurs qui vont permettre d'indexer les flux de données à partir de diverses sources (Mysql, MongoDB, flux rss, csv ...) .

Un river est composé d'un nom unique et d'un type (à noter que plusieurs rivers du même type, mais de noms différents, peuvent être lancés au sein d'un même cluster).

Les rivers sont dans le cluster et chacun d'eux est alloué automatiquement à un nœud et lancé. Par conséquent, si un nœud est défectueux, le river est automatiquement réaffecté sur un autre nœud.

Dans le cadre de ce projet, nous avons choisi d'utiliser le river CSV. Il s'agit en effet, d'un plugin qui permettra de déverser les documents en continu dans ElasticSearch.

Il suffira donc, de mettre en place une routine qui alimentera le dossier à partir duquel le river csv viendra s'alimenter.

Pour installer le river-csv, il faut télécharger le fichier à partir de l'URL suivante :

<https://github.com/AgileWorksOrg/elasticsearch-river-csv/releases/download/2.1.1/elasticsearch-river-csv-2.1.1.zip>

Ensuite, il faut simplement aller dans le dossier bin/plugin -install river-csv

```
curl -XPUT localhost:9200/_river/my_csv_river/_meta -d '{
  "type": "csv",          <----- Ici mettre le type de fichier à importer
  "csv_file": {
    "folder": "/tmp",    <----- Ici mettre l'endroit où se trouvent les CSV à indexer
    "first_line_is_header": "true" <----- Ici la première ligne est le nom des colonnes
  }
}'
```

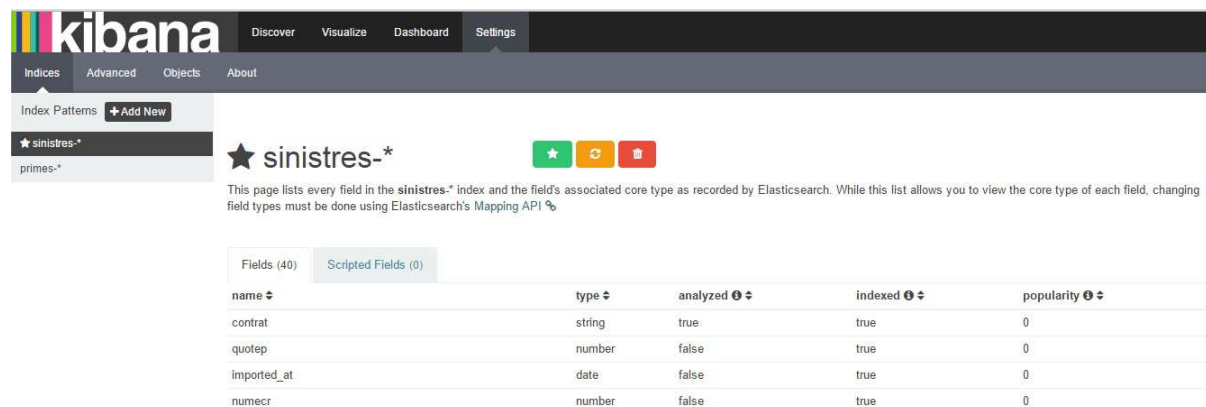
F) L'analyse des données avec Kibana

Kibana via l'interface web va permettre à n'importe qui de pouvoir sortir des tableaux de bord sur les primes et sinistres juste à partir de l'onglet **Dashboard**.

Après avoir démarré Kibana, il faut taper l'URL suivante : <http://localhost:5601>

Ensuite, il faut entrer le nom de l'index recherché, par exemple **primes-allianz** ou **sinistres-allianz** dans notre cas.

Dans l'onglet **Settings**, on trouve l'index avec les variables ainsi que leurs types définis par le mapping



The screenshot shows the Kibana interface with the 'Settings' tab selected. The left sidebar shows the 'Indices' section with 'sinistres-*' selected. The main content area displays the settings for the 'sinistres-*' index. A table lists the fields and their associated core types and settings.

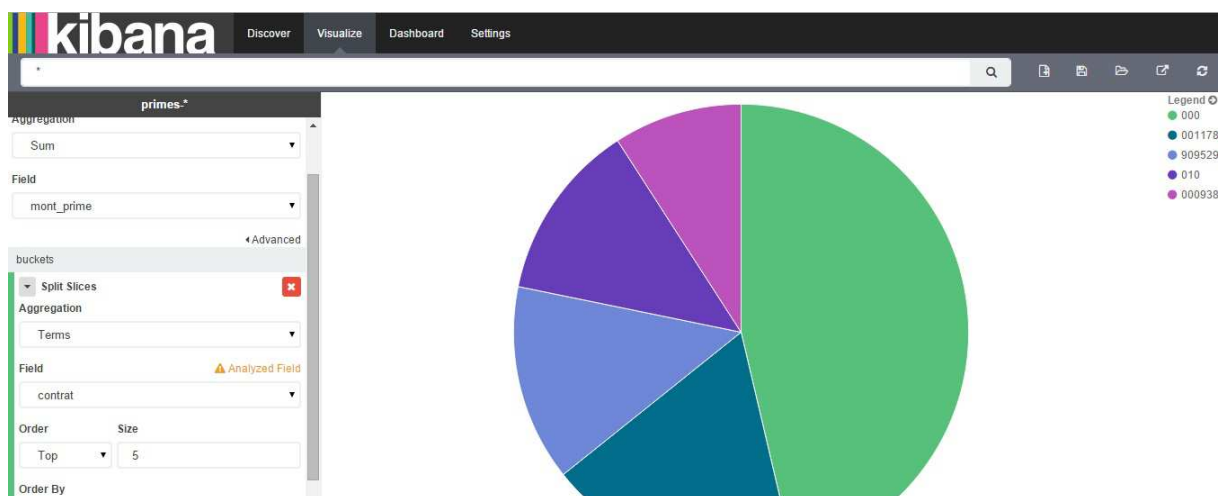
name	type	analyzed	indexed	popularity
contrat	string	true	true	0
quotep	number	false	true	0
imported_at	date	false	true	0
numecr	number	false	true	0

L'onglet **Discover** permet le parcours des données :

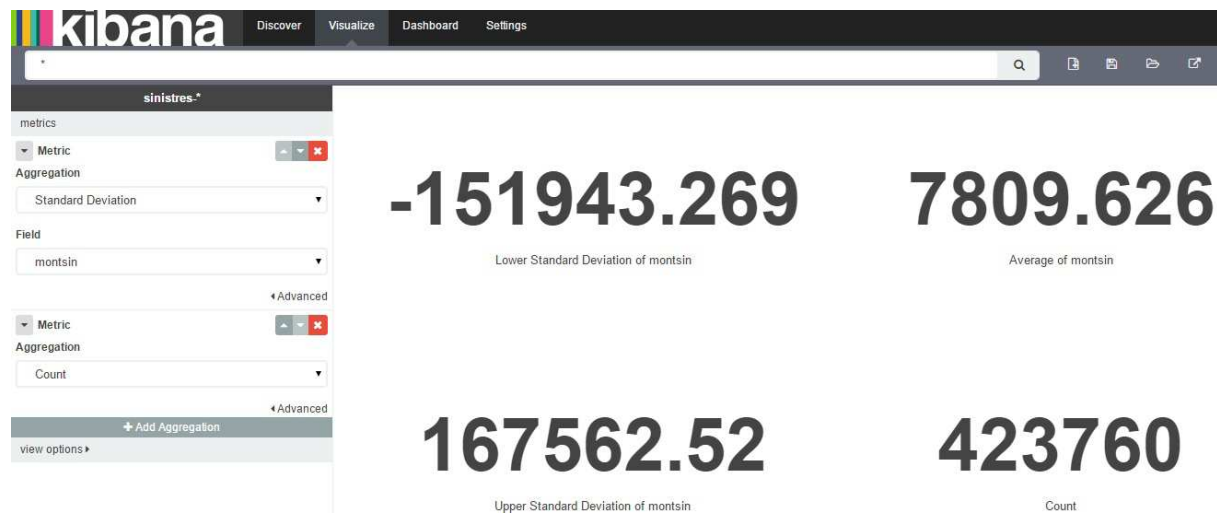


L'onglet **Visualize** permet une analyse rapide des données en les présentant sous forme de diagrammes à secteurs circulaires, graphes ou encore juste des nombres de sinistres dans l'année pour tel contrat etc....

Si l'on veut faire une recherche pour connaître en temps réel les contrats les plus importants, avec un diagramme à secteurs circulaires par exemple.



La partie metric va permettre de compter le nombre de lignes, de connaître le montant moyen de sinistres, l'écart type, de connaître le montant minimum et maximum de sinistres.



L'avantage de Kibana est que les données peuvent être affichées dans un tableau de bord de manière visuelle. Il est très facile d'ajouter un panel et le sauvegarder.

Ensuite on peut choisir le type de graphique (ici « terms » de type « bar » pour l'histogramme ou « pie » pour les diagrammes à secteurs circulaires).

Pour notre problématique, il sera possible de créer des Dashboards prédéfinis.

[G\) Implémentation des facettes](#)

ElasticSearch, depuis sa version 1.0.0, a rendu disponible un tout nouveau module appelé Aggregation.

Celui est censé remplacer, à terme, le module en charge de la gestion des facettes.

Dans un premier temps, nous commencerons par présenter les facettes puis finirons avec une présentation du module Aggregation.

Les résultats donnés par ElasticSearch peuvent être remontés par des facettes.

Les facettes sont des agrégats des résultats de recherche. A partir de ces facettes, l'utilisateur peut alors affiner sa recherche en ayant un visuel sur les résultats qui seront ramenés.

Le principal objectif d'un moteur de recherche "full-text" est de retourner un nombre limité de documents répondant à une requête.

Les facettes permettent donc l'agrégation des résultats d'une recherche et vont s'adapter aux résultats de recherche de l'utilisateur, celui-ci pourra alors affiner sa recherche.

Les facettes disponibles sont les suivantes :

- Range : permet de spécifier des intervalles de données
- Histogram fonctionne avec des données numériques par la construction d'un histogramme à travers les intervalles des valeurs du champ
- Date Histogram est une facette permettant des histogrammes sur des champs date
- Filter est une facette qui permet de retourner les recherches les plus populaires
- Query permet de retourner un nombre de hits correspondant une requête
- Statistical : est une facette qui nous intéresse car elle nous permettra de calculer des données statistiques sur un champ numérique. Les données statistiques incluent le décompte, le total, la somme des carrés, la moyenne, le minimum, le maximum, la variance et l'écart type
- Terms Stats combine à la fois les facettes term et statistical.
- Geo Distance : permet de fournir les données correspondant à un intervalle de distance géographique

Agrégation

Ce concept recouvre celui des facettes tout en offrant, par ailleurs, une solution aux limitations que l'on pouvait rencontrer dans la mise en œuvre des facettes. Nous reviendrons sur ces limitations mais, commençons tout d'abord par présenter ce que sont les agrégations.

Il existe deux types d'agrégations : celle dites metrics et celles dites bucketing.

Les agrégations de type metrics

```
curl -XGET http://localhost:9200/sinistres-allianz/_search -d '{"aggs":
  {"montant-sin-ag":
    {"stats":
      {"field": "montsin"}
    }
  }
}'
```

Cette requête nous renverra les valeurs suivantes : min, max, sum, count et avg pour le champ défini, ici «montsin»

Ce type d'agrégation (min, max, sum, count, avg) s'emploie uniquement sur des champs numériques. Pour obtenir le max pour le champ montsin pour les sinistres nous pourrions exprimer la requête comme suit :

```
{"aggs":
  {"montant-sin-min":
    {"min":
      {"field": "montsin"}
    }
  }
}
```

```
}  
}
```

Les agrégations de type buckets

Ces types d'agrégations sont similaires aux facettes tout en étant plus puissantes et plus simple à mettre en œuvre.

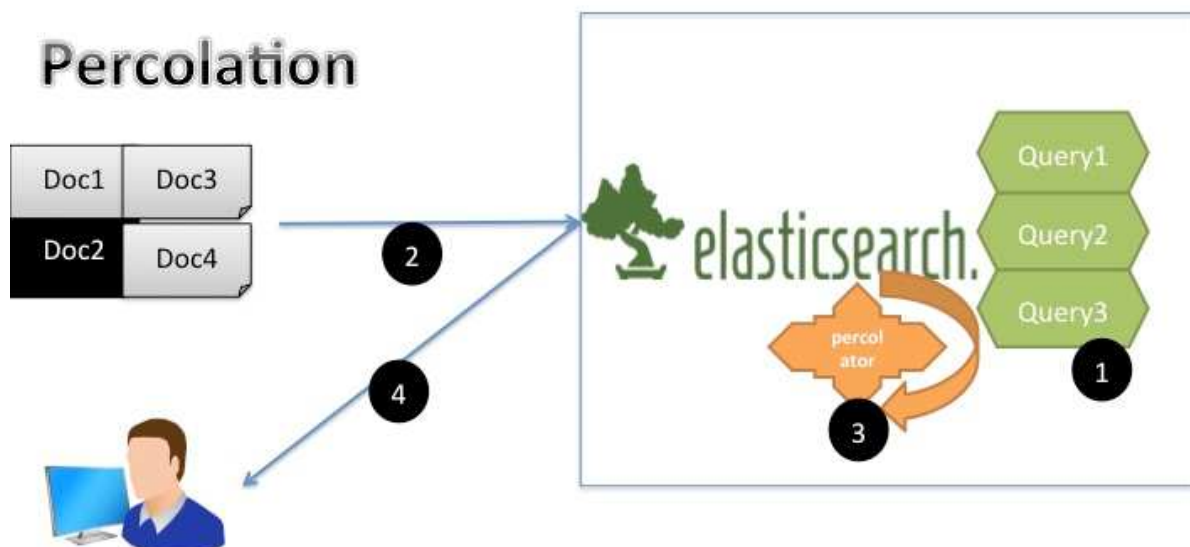
Ce type d'agrégation est lui-même composé de sous-types tels que : terms, range, date_range, ipv4 range ...

Au-delà de la simplicité de sa mise en œuvre et de sa puissance, le moteur d'agrégation, contrairement à celui des facettes, présente l'énorme avantage d'une imbrication (des agrégations) de manière quasiment infinie.

IV) Percolation

La percolation consiste en la possibilité de rechercher des documents correspondant à un document. A ce titre, elle est aussi appelée recherche inversée.

En effet, pour ce faire, on indexe non plus seulement les documents mais aussi les requêtes, puis on soumet des documents à Elasticsearch en lui demandant de nous renvoyer les requêtes correspondantes.



1. les requêtes « query1 », « query2 », « query3 » sont indexées et stockées dans elasticsearch au même titre que n'importe quel document.
2. On « push » les documents dans elasticsearch
3. Le percolateur essaye de mettre en relation les documents avec les requêtes sauvegardées
4. Elasticsearch renvoie le résultat du processus de percolation: les recherches associées au document fourni.

Exemple de mise en œuvre

- Indexation de la requête contenue dans le fichier query1.json

```
curl -XPUT 'http://localhost :9200/sinistres-allianz/.percolator/1' -d @query1.json
```

- Soumission d'un document au percolator

```
curl -XGET 'http://localhost :9200/sinistres-allianz/sinistres/_percolate ?pretty' -d @sinistre.json
```

La réponse d'élasticSearch, dans l'hypothèse où le document correspondrait à la réponse pré-enregistrée, pourrait ressembler à ceci :

```
'matches' :{  
'index' : 'sinistres-allianz',  
'_id' : '1'  
}
```

Intérêt de la percolation pour notre solution

Dans le cas de la solution que nous sommes en train de décrire, la percolation pourrait, par exemple, permettre de déceler les données à la marge ou aberrantes ou incohérentes du point de vue du système étudié.

IV) Scalabilité

Lors de l'indexation d'un document, celui-ci est envoyé sur le nœud maître, il lui est automatiquement alloué une partition puis il est répliqué.

ElasticSearch fait une réallocation dynamique des partitions qui se fait par hachage ; il va ensuite se charger de la réallocation de ces partitions sur chaque nœud pour diminuer la charge de chacun des nœuds.

Ci-dessous un exemple ; supposons que nous ayons 3 nœuds avec 4 partitions au départ, nous avons la visualisation suivante :



L'ajout d'un nouveau nœud va donner une redistribution des partitions de la manière suivante afin de diminuer la charge



IV) Sécurité

Etant donné la nature des données ainsi que le contexte d'implantation de la solution, il nous paraît crucial d'adresser de façon minutieuse la sécurisation des données. Ici, par sécurité nous nous intéressons surtout à garantir la confidentialité des données.

ElasticSearch propose, sous licence commerciale, un produit nommé nommé Shield qui offre les fonctionnalités suivantes :

- Gestion de l'authentification : l'authentification s'appuiera sur l'annuaire LDAP de l'entreprise. Il ne s'agira pas seulement d'identifier la personne comme étant employée chez Allianz mais il faudra aussi garantir son rattachement au service chargé de gérer les primes et les cotisations.
- Gestion de l'autorisation : Ici, il s'agira d'accorder des droits aux personnes authentifiées en fonction de leurs profils.
En effet, par défaut, toute personne ayant accès à l'interface http (REST) d'Elasticsearch est en droit non seulement d'interroger n'importe quel index hébergé par ce dernier mais aussi, d'y réaliser des tâches d'administration.
Pour ce faire, nous mettrons donc en place un mapping entre les rôles déclarés dans ElasticSearch et les groupes LDAP / active directory.
- Cryptage ssl des échanges entre le poste de l'utilisateur et le endpoint REST d'elasticsearch d'une part et, entre les différents nœuds du cluster d'autre part.

- La solution n'est accessible qu'en https.
- La communication entre les différents nœuds du cluster elasticsearch se fera via des canaux cryptés utilisant le protocole ssl.
- L'accès à l'application nécessitera une authentification à un annuaire LDAP ou active directory.



Cryptage des échanges au sein d'un cluster Elasticsearch grâce au plugin Shield